

TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

ROBOTAKSİ – BİNEK OTONOM ARAÇ YARIŞMASI

KRİTİK TASARIM RAPORU

(HAZIR ARAÇ KATEGORİSİ)

TAKIM ADI: VOLANS

TAKIM ID:459209

İÇERİK

1. Takım Organizasyonu	3
1.1. Özet	3
1.2. Organizasyon Şeması	3
2. Ön Tasarım Raporu Değerlendirmesi	3
3. Hazır Araç Özellikleri ve Analizi	4
3.1. Aracın Teknik Özellikleri	4
3.2. Velodyne VLP-16 Puck Hi-Res Lidar	5
3.3. Zed 2 Kamera	5
3.4. NVIDIA Jetson AGX Xavier	6
4. Araç Kontrol Ünitesi	6
4.1. CAN BUS	6
4.2. PID	7
5. Araç Dinamiği Modelleme/Tanımlama ve Kontrolü	7
6. Otonom Sürüş Algoritmaları	8
6.1. Şerit Takibi Algoritması	8
6.2. Nesne Tanımlama Algoritması Ve Nesne Uzaklık Tespiti Algoritması	8
6.3. Park ve Durak Algoritması	11
7. Güvenlik Önlemleri	13
7.1. Dönüş Güvenlik Önlemleri	13
7.2. Şerit Tespiti Güvenlik Önlemi	13
7.3. Nesne Takibi Güvenlik Önlemleri	13
7.4. Donanımsal Güvenlik Can Protokolü	13
7.5. Araç üzerindeki güç kesiciler	13
8. Simülasyon	14
8.1. Gazebo	15
8.2. ROS	15
8.3. Simülasyon Ortamı	15
9. Sistem Entegrasyonu	16
10. Test ve Doğrulama	16
11. Referanslar	17

1. Takım Organizasyonu

1.1. Özet

Takımımız Volans, Hasan Kalyoncu Üniversitesi Mühendislik Fakültesi öğrencileri tarafından oluşmaktadır. Son durumda takımımızda 2 Elektrik–Elektronik Mühendisliği, 1 Bilgisayar Mühendisliği ve 4 Yazılım Mühendisliği öğrencisi olmak üzere toplamda 7 tane lisans eğitimi alan öğrenci bulunmaktadır. Amacımız doğrultusunda yazılımsal ve donanımsal anlamda ihtiyacımız olan materyaller belirlendikten sonra takım üyelerimiz görev dağılımı yapıp çalışmalarına başlamıştır. Üyelerimiz düzenli aralıklarla online veya yüz yüze olacak şekilde bir araya gelerek projemizin sürecini takip etmiş, kendileri arasında bilgi alışverişi yapmış, aktif bir şekilde projemizde ilerleme kaydetmiş bulunmaktadır. Projemiz ile ilgili çalışmalar ekiplerimizin birbirleriyle koordine şekilde ilerlemesiyle devam etmektedir.

1.2. Organizasyon Şeması

Şerit Takibi	Nesne Tanımlama	Kontrol ve Test
• Çeşitli filtrasyonlar kullanılarak aracın şeritler arasında kalması sağlandı. • Sefa Teke • Lale Cemile Bozkurt	• Parkurda zed ve telefon kamerasıyla çekilen fotoğrafları Roboflow ile birlikte etiketleyip, yolov5 ve pytorch ile veri seti eğitildi. • Didar İlayda Çiçek • Cihan Çivit	• Araç parkur ve parkurdaki görevi konusunda algoritmalar yazılarak tasarlandı. • Beyza Nur Utuş • Furkan Dinç • Yaren İyitütüncü

Tablo 1

Yukarıdaki şemada her bir takım arkadaşımızın sorumlu olduğu görevler belirtilmiştir. Görev dağılımı sonucu üyeler kendi konu başlıkları ile ilgili araştırma ve incelemelerde bulunup çalışmalara başlamışlardır. Bu çalışmalar birbirlerine uyumlu şekilde sürmektedir. Araçla ilgili analizler yapılarak yazılım tarafında algoritma çalışmaları devam etmektedir.

2. Ön Tasarım Raporu Değerlendirmesi

Hakemlerin incelemesi sonucu eksiklerimiz olmasına rağmen Ön Tasarım Raporumuz geçer not almıştır. Değerlendirme raporunun tarafımıza iletilmesi ile eksiklerimiz belirlenmiştir. Bu çalışmalarda hedefimiz yazılımımızı daha iyi hale getirmektir. KTR’de aracın özellikleri Ön Tasarım Raporuna göre daha detaylı şekilde ele alınıp araç analizi ve özellikleri anlatılacaktır. Aracın üzerindeki sensörlerin özellikleri, otonom sürüş algoritmaları, araç kontrol ünitesi, güvenlik önlemleri ve en son olarak sistem entegrasyonu hakkında bilgiler için derinlere inilip her yönüyle ele alınacaktır.

Ön Tasarım Raporunda nesne tanımlama için Yolov3 kullanacağımızı belirtmiştik. Son durumda Yolov5 üzerinde çalışılmıştır. Aynı zamanda takımdan bir arkadaşımız ayrılmış olup, son durumda takımımız 7 kişiden oluşmaktadır.

3. Hazır Araç Özellikleri ve Analizi

TEKNOFEST Komitesi tarafından yarışmada kullanılacak tam donanımlı elektrikli araç platformunun teknik özelliklerine aşağıdaki tabloda yer verilmiştir.

3.1. Aracın Teknik Özellikleri

Teknik Verileri

Motor	7.4 KW Schaubmuller
Tork/ Nm (Max)	125 NM
Çalışma Gerilimi /V	48 Volt
Batarya	Trojan T145 Plus
Oturma Kapasitesi	2,4,6,8
Max Araç Hızı Km/h	24
Şasi	6000 Serisi Yüksek Mukavemetli Uçak Sınıfı Alüminyum

Tablo 2

Süspansiyon Sistemi

Ön Süspansiyon	Bağımsız Teleskopik Amortisör
Arka Süspansiyon	Helezon Yaylı Sabit Aks/ Teleskobik Amörtisör
Lastikler	10" Duro

Tablo 3

Fren Sistemi

Park Freni	Mekanik/Ayak Pedallı
Servis Freni	Hidrolik/Kampana

Tablo 4

Araç Parçaları ve Modelleri

Oriental Motor	BLVM640N-GFS_ORIENTAL
Manyetik Kavrama	ABK 04 3.1 DIZAYN
Lineer Aktuatör	MD24A050-0100CNO2NNSD
Enkoder	8.F5888.5B2F.2123
Lidar	Velodyne Puch Hi-Res 16
IMU	https://www.sbg-systems.com/products/ellipse-micro-series/
Otonom Kontrolör	Nvidia Xavier
Araç Kontrolörü	Otomotiv Araç Kontrol Ünitesi § 3 x Canbus § 1 x GPS § 1 X RS485 § 1x GSM Modülü

Tablo 5

Aracın Ölçüleri

Dingil Mesafesi	2400	Toplam Yükseklik (Konopisiz)	1240
Toplam Uzunluk	2385	Toplam Yükseklik (Konopili)	1950
Toplam Uzunluk (Geri oturma aparatıyla birlikte)	2825	Yükleme Zemini Yerden Yükseklik	680
Ön Uzunluk	290	Toplam Genişlik (Konopisiz)	1330
Arka Uzunluk	595	Arka İz Genişliği	1115
İlave Ters İki Koltuklu Ölçü	1035	Ön İz Genişliği	1030
Şasi Yerden Yüksekliği (Konopisiz)	185		

Tablo 6

Aracın üzerinde yer alan sensörlerle ve yazılımsal eklentilerle beraber otonom sürüş algoritması belirlenmiş olup bu bölüm için 6. Otonom Sürüş Algoritmalarında detaylı bilgiler verilmiştir.

3.2. Velodyne VLP-16 Puck Hi-Res Lidar

Lidar, bir mekanizmasının lazer tarama çekirdeği ve çekirdeğini yüksek hızda döndürmesi prensibi ile çalışmaktadır. Bu mekanizma yüksek dönüş hızıyla ve seri lazer atışları yapabilmesiyle (saniyede 1000 sayısına ulaşabilmektedir.) çevresindeki engelleri ve önemli nesneleri algılama işlemini gerçekleştirir. Teknofest Robotaksi-Otonom Araç Yarışması, hazır araç kategorisi kapsamında sunulan araçta Velodyne Puck Hi-Res lidar bulunduğu için çalışılan simülasyon ortamına Velodyne Puck Hi-Res lidar entegre edilerek kullanılmıştır.



Resim 1

3.3. Zed 2 Kamera

ZED2 dünyanın en hızlı derinlik kameralarından biri olduğundan ve araca 120 derecelik bir görüntü açısı sağladığından ve ayrıca otonom sürüşün gerçek dünyada, gerçek zamanlı olarak kullanılacak olduğundan görüntülerin yüksek doğrulukla, yüksek çözünürlükle ve en hızlı şekilde araca ulaşması gerekmektedir. Zed 2 kamerasının gerçek zamanlı derinlik tabanlı görsel kilometre sayacı teknolojisi ile görüntü en optimum şekilde araca iletilmektedir. Aynı zamanda çoklu sensör kullanımı için en uygun kamera özellikleri taşımaktadır. Böylece GPS, IMU gibi sensörlerle füzyonu kolay hale gelmiştir.

Teknofest Robotaksi-Otonom Araç Yarışması kapsamında bu kameranın diğer kameralara göre çok daha makul olduğu açıkça görülmektedir.



Resim 2

Şeritlerin ve engellerin tanınması, aracın konumunu ve çevresini algılayıp verilen otonom görevi yerine getirmesi kamera veya başka sensörler aracılığıyla yapılacaktır. Zed 2 kameradan alınan gerçek zamanlı, yüksek çözünürlüklü görüntü verileri işlenerek; Yolo yöntemleri kullanılarak en optimum füzyon oluşturulmaya çalışılmıştır.

3.3. NVIDIA Jetson AGX Xavier

Teknik özellikler:

- Dönüş hızı: 5 – 20 Hz
- Hassasiyet: +/- 3 cm (ortalama)
- Kolay konfigürasyon için dahili web server
- 100 metreye kadar efektif mesafe
- 16 kanal

4. Araç Kontrol Ünitesi

4.1. CAN BUS

Aracımız kablosuz kumanda ile kontrol edilebilmektedir. CANBUS iki kablo hattı üzerinden birçok birimin haberleşmesini sağlayan bir haberleşme protokolü olarak aynı zamanda araca komut vererek hareket ettirmemizde mümkündür.

Araç kontrolünü sağlamak için özet bilgiler aşağıdaki tabloda yer almaktadır.

Name	Message	Start Bit	Lenght Bit	Initial Value	Factor	Offset	Minimum	Maximum
Operational	VEHICLECTRL	0	1	0	1	0	0	0
Steering	VEHICLECTRL	8	8	0	1	1	-128	127
Angle								
Speed	VEHICLECTRL	16	8	0	1	1	-128	127
Brake	VEHICLECTRL	24	8	0	1	0	0	0

Tablo 7

Operasyonel, elektrik motorunu etkinleştirmek için kullanılır ve aracı sürmek için 1 olabilir. Direksiyon açısı 0 ile 255 arasında değişir. Maksimum sola dönüş için 0, maksimum sağa dönüş içinse 255 göndermek gerekir. Bu nedenle düz hareket 128 göndermeyi gerektirir. Araca CAN BUS haberleşme protokolü üzerinden direksiyon açısı, ileri-geri gaz değeri ve fren değeri gönderilebilmektedir. Bu değerler 16'lık sayı tabanı olan hexadecimal türünden gönderilmektedir.

İleri-geri gaz değeri 00-FF aralığındadır. 00 değeri tamamen aracı geriye doğru hareket ettirmektedir. FF değeri ise aracı tamamen ileri doğru hareket ettirmektedir. Direksiyon açısı değeri 00-FF aralığındadır. 00 değeri direksiyonu tamamen sola çevirmektedir. FF değeri ise direksiyonu tamamen sağa çevirmektedir. Direksiyon için offset değeri 80 olarak verilmiştir. Fren değeri 00-FF aralığındadır. FF değeri araca tamamen fren yaptırmaktadır.

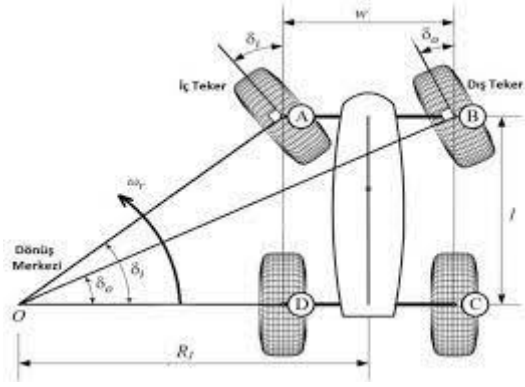
Aracı AGX Xavier ile kontrol etmek istediğimizde USB portu üzerinden ZED2 kamerasının Ros başlıklarını, Xavier'in CAN alıcısı üzerinden IMU ve GPS verilerini baz almamız gerekmektedir. Ayrıca araçla gezinmek için istenilen hız ve yönlendirme komutları da aynı şekilde gönderilmelidir.

4.2. PID

Araç kontrol ünitesi gömülü PID algoritmasını uygulayarak hız kontrolünü sağlayabilmektedir. Bu nedenle motor kontrolünü uygulamak için sadece hız bilgisini istemektedir. PID aslında bir kontrol geri bildirim mekanizmasıdır. Hata değerlerini hesaplayıp, hatanın en aza indirilmesi sağlanır. PID Algoritması K_p , K_i ve K_d katsayılarından oluşur. Bu katsayılar deneme yanılma yöntemiyle hesaplayıp algoritmada yerini almıştır.

5. Araç Dinamiği Modelleme/Tanılama ve Kontrolü

Ackermann yönlendirme geometrisi araç modelinin hareketi için kullanılmıştır. Bu geometri tekerlek üzerinde statik olarak tanımlı "Toe" açısının düzenli olarak değişmesi sağlanır. Bu algoritmanın yapıma kullanılma sebebi aracın üst kısımdan bakıldığında daha net şekilde görüldüğü üzere ön tekerleri ile arka tekerlerinin aynı görüngede yol almamasıdır. Ön tarafın arkaya göre kapalı olmasına toe-in, açık olmasına toe-out denir. Ackermann yönlendirme geometrisi sayesinde direksiyon döndürüldüğünde hangi tekerleğin kaç derecelik bir açı yapması gerektiğini hesaplamamızı sağlar, böylelikle her bir ön tekerleğin birbirinden bağımsız bir şekilde dönmesi sayesinde aracın dönerken kaymaması sağlanır. Düzlemsel hareket yapan aracın hareketi esnasında her tekerleği bir yay üzerinde hareket etmektedir. Dönüş yapılırken aracın kaymasını engellemek, aracın dönüş kararlılığını korumak ve minimum tekerlek aşınmasını önlemeyi sağlar. Bunun için de tekerleklerin taradıkları yayların merkezinin yani ani dönme merkezlerinin çakışık olması koşulundan geçer ,bu duruma Ackermann geometrisi denir. Ackermann geometrisindeki her tekerleğin merkez çizgisi ile arasındaki mesafe (T), aracın dingil mesafesiyle olan mesafe (L), tekerleğin dümdüz ileriye olan açısı (α_1), dış tekerleğin düz önden açısı (α_0) ve dönüş yarıçapı ise (R) ile formüle geçirilir. Ackermann Geometrisi'nin Üstten Görünümü Sabit bir hızda yalnızca ön tekerleklerin yönlendirildiğini varsayarsak, tekerleklerde ideal açıları bulmak için aşağıdaki formül kullanılır.



$$\alpha_i = \tan^{-1} (L/R - T/2) \quad \alpha_o = \tan^{-1} (L/R + T/2)$$

Resim 3

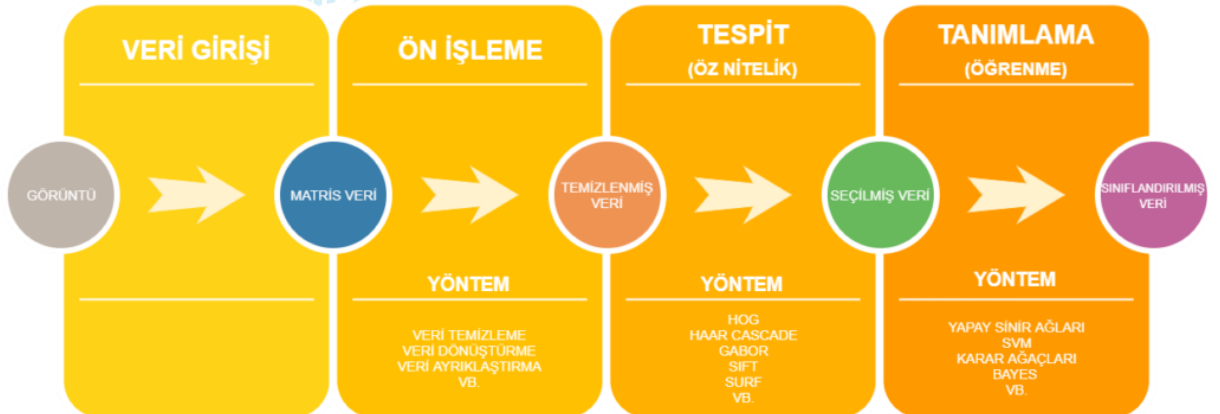
Ackermann Dönüş Grafiği Kırmızı çizgi, basit dönüş için iki tekerlek açısı arasındaki ilişkiyi gösterir. Mavi çizgi, ideal dönüş için iki tekerlek açısı arasındaki ilişkiyi gösterir. Dönüş ne kadar sıkı olursa, iç açının dış açıya oranı o kadar yüksek olur. Tekerlekleri bu şekilde döndürmek için bir mekanizmanın nasıl oluşturulacağı konusundaki zorluğu, Wilhelm Ackermann yine kendi soyadıyla isimlendirdiği Ackermann geometrisi ile çözmüştür.

6. Otonom Sürüş Algoritmaları

6.1. Şerit Takibi Algoritması

Şerit Takibini görüntü işleme yönteminden (OpenCv kütüphanesinden) yararlanarak Zed 2 kamerası üzerinden yaptık. Gri maskeleme yaparak renkleri sadeleştirdik. Beyaz maskelemeyle beyaz dışındaki koyu renkleri ayırdık. Kenar tespiti yapabilmek için Gaussian blur filtresini kullanarak görüntüyü bulanıklaştırdık. Sonrasında Canny Edge Dedector kullanarak kenarları çizdik. Şeritler dışındaki nesneleri kamera algıladığı için bunları filtrelememiz gerekti. Bunun içinde Region of Interest (ROI) kullanarak kameranın sadece şeritleri gördüğü kısımlarla sınırlandırdık. Kesikli görünen şeritleri kullandığımız kodlarla birleştirdik ve belirledik. Neticede Zed 2 kameradan gelen görüntüyle şerit takibi yapabilmekteyiz.

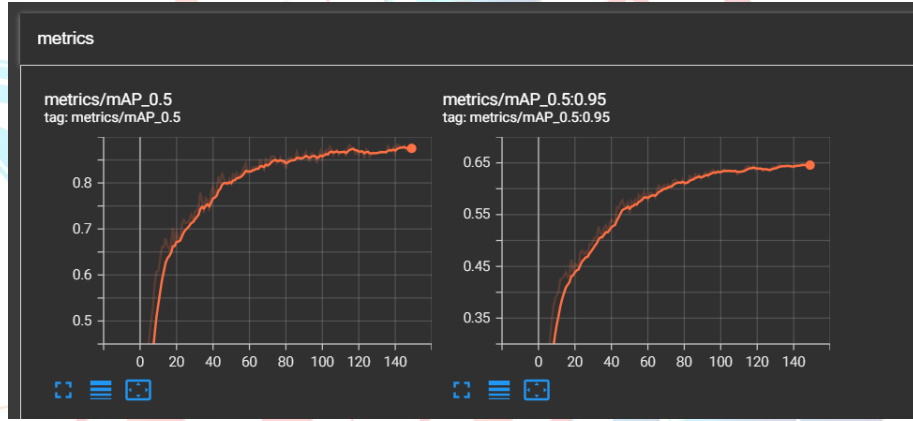
6.2. Nesne Tanımlama Algoritması Ve Nesne Uzaklık Tespiti Algoritması



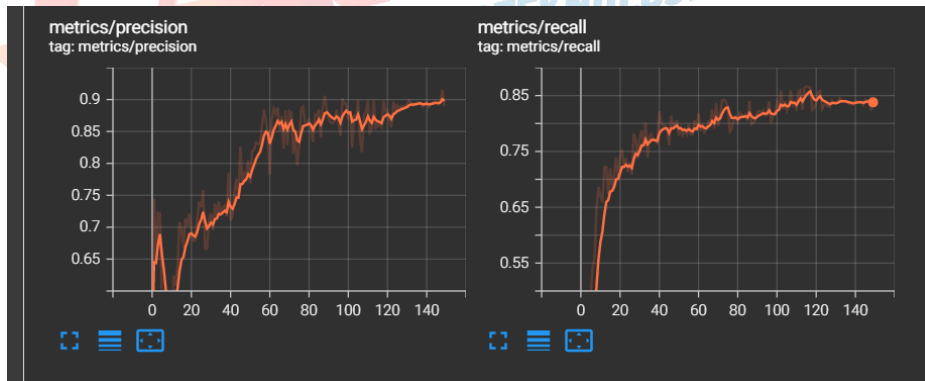
Resim 4

Zed 2 kamerasından gelen görüntülerden trafik ışıkları ve levhalarını tanıyarak yön değiştirme, durakta durma, bekleme park etme gibi faaliyetleri yapabilmek için geliştirdiğimiz nesne tanımlama algoritmamız nesne uzaklık tespiti algoritması ile ortak çalışarak kamera açısına birden fazla levha veya trafik ışığı girebileceğinden yakın olan levhayı algılamaktadır. Nesne tanımlamanın ilk aşaması olarak gerek alanda kendi imkânlarımız ile çektiğimiz gerekse aracın üzerinde bulunan Zed 2 kamera ile çektiğimiz ve internet üzerinden bulduğumuz fotoğraflar ile veri girişi gerçekleştirilmiştir.

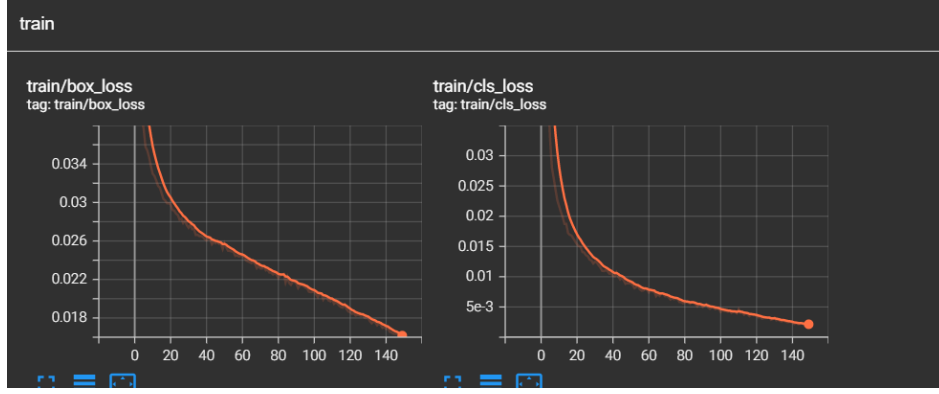
Levha ve Trafik Işıkları Tanıma Trafik işaretlerinin tespiti üzerine araştırma sırasında derin öğrenme metodu ile Yolo'nun 5. versiyonu ve Pytourtch en optimize seçenek olarak görüldüğünden bu seçenek üzerine düşülmüştür. Daha sonra bu fotoğrafları matrislerine ayırarak bir ön işleme aşamasından geçirilerek veri setimiz elde edilmiştir. Bu veri seti üzerinde bazı filtreler uygulayarak Zed 2 kameradan alınan girdilerde bulunabilecek hatalara karşı veri setimizi çeşitlendirilmiştir. Bu veriler YOLOv5 mimarisine uygun olarak tam tamına 18 farklı trafik tabelası ve trafik ışığı etiketlenmiştir ve birden fazla kez eğitilmiştir. Resim 5 Resim 6 ve Resim 7 'de görüldüğü gibi veri setimizi eğitimi sonucu tensorboard yardımıyla elde ettiğimiz grafikler verilmiştir.



Resim 5



Resim 6



Resim 7

Model yarışma alanında çekilmiş olan video üzerinde ve canlı veriler ile test edilmiş olup başarı oranının %90 civarlarında olduğu görülmüştür. Ancak en büyük problem ise nesne tespiti esnasında levhalara ve trafik ışığına olan uzaklığa bakılmaksızın algoritmaların direkt işlevini yerine getirmesiydi. Bunun önüne geçmek için tespit edilen levhanın veya trafik ışığının araca olan uzaklığı, derinlik kamerasından hesaplanır ve her levha veya trafik lambası için devreye girme mesafesi farklı olması kararlaştırılmıştır. Hesaplanan uzaklık verisinin belirlenen sayıya ulaşmasıyla levhaların doğruluk değeri kontrolüne başlanır. Bu iki değerde algoritmanın tetiklenmesi için yeterli ise algoritma çalışmaya başlamaktadır.



Resim 8



Resim 9



Resim 10



Resim 11

Lidar Sensörü araç ile engel arasındaki uzaklıkları hesaplayarak engellerle arasındaki mesafeyi korumaktadır. Zed 2 kamerasının algılayamadığı veya algılayamayacağı durumlarda güvenilir bir veri üreterek aracı kazalardan önlemekte başarılı olmuştur. Zed 2 kamera ve Lidar sensör verileri aracın karar verme algoritmalarını sağlayacaktır. Araç aldığı girdilere göre hız, fren ve direksiyonunun açısı değerini çıktı olarak göndermektedir.

Zed 2 kamera ile algılanan Durak levhası için öncelik nesne tanımlama ile senkronize çalışan mesafe algoritmasıdır. Mesafe yeterli değere ulaştığı anda durak algoritması etkin hale gelmektedir.

Ve belirlenen miktarda sağ yönde hareket ettikten sonra sol hareketini aracı düzeltmek için yapar. 30 saniye bekleme kuralı olan durak görevi için ayarlanan bekleme süresi 40 saniyedir. 40 saniyenin ardından sol hareketine başlar ve belirlenen süreden sonra sağ yapıp aracı düzenler. Daha sonra diğer algoritmalar tekrardan aktive olur ve araç normal düzenine döner.

Zed 2 kamera ile algılanan park levhası için öncelik yine nesne tanımlama ile çalışan mesafe algoritmasıdır, temelde park algoritmasından farkı şerit takibi algoritmasını düzenli olarak çalıştırmasıdır. Park levhasını mesafe kontrolü ile düzenli olarak takip eder ve belirlenen mesafenin altına düşüldüğü anda araç durur.

6.3. Park ve Durak Algoritması

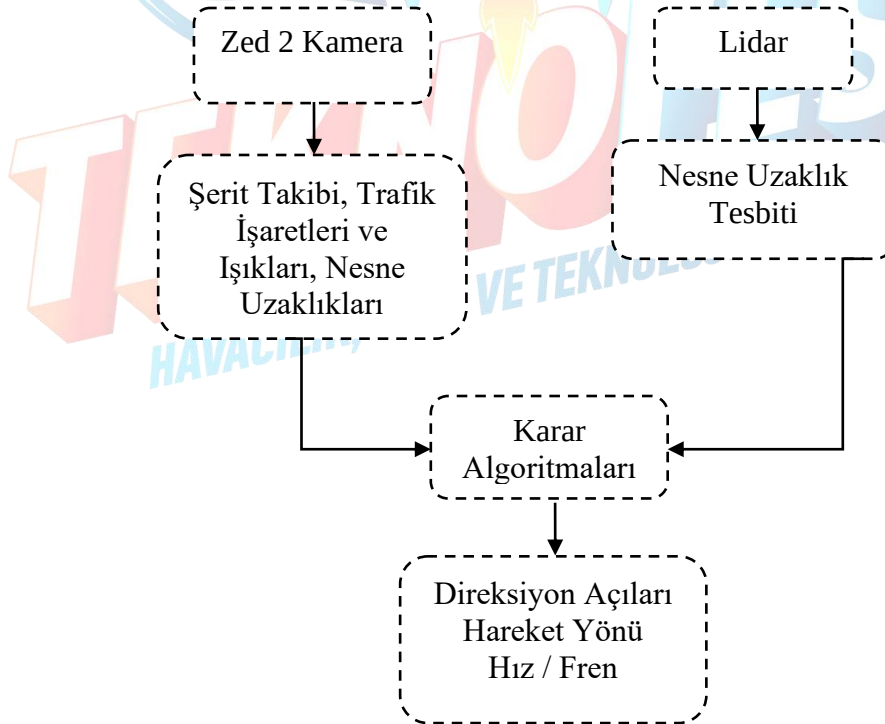
Bu algoritmaların bilgi girişleri Zed 2 kamera ve Lidar sensörüdür çıktısı ise Hız - Fren - Hareket Yönü Direksiyon Açılarıdır.

Sensörlerden alınan verileri gerekli işlemleri uyguladıktan sonra Can protokolü ile dönütler verilmektedir. Zed kamerasından gelen görüntülerden trafik ışıkları ve levhalarını tanıyarak yön değiştirme, durakta, bekleme park etme gibi faaliyetleri yapmaktadır ayrıca oluşturduğumuz algoritma mesafe hesabı da yapabildiğinden bir güvenlik önlemi olarak 1 metre mesafe içinde bir nesne bulunduğunda aracı durdurmaktadır ve ayrıca kamera açısına birden fazla levha veya trafik ışığı girebileceğinden yakın olan levhayı algılamasını algoritmamız içine ekledik.

Lidar Sensörü araç ile engel arasındaki uzaklıkları hesaplayarak engellerle arasındaki mesafeyi korumaktadır. Zed 2 kameranın algılayamadığı veya algılayamayacağı durumlarda güvenilir bir veri üreterek aracı kazalardan önlemekte başarılı olmuştur.

Zed 2 kamera ile algılanan Durak levhası için öncelik nesne tanımlama ile senkronize çalışan mesafe algoritmasıdır. Mesafe yeterli değere ulaştığı anda durak algoritması etkin hale gelmektedir. Ve belirlenen miktarda sağ yönde hareket ettikten sonra sol hareketini aracı düzeltmek için yapar. 30 saniye bekleme kuralı olan durak görevi için ayarlanan bekleme süresi 40 saniyedir. 40 saniyenin ardından sol hareketine başlar ve belirlenen süreden sonra sağ yapıp aracı düzenler. Daha sonra diğer algoritmalar tekrardan aktive olur ve araç normal düzenine döner.

Zed 2 kamera ile algılanan park levhası için öncelik yine nesne tanımlama ile çalışan mesafe algoritmasıdır, temelde park algoritmasından farkı şerit takibi algoritmasını düzenli olarak çalıştırmasıdır. Park levhasını mesafe kontrolü ile düzenli olarak takip eder ve belirlenen mesafenin altına düşüldüğü anda araç durur.



Tablo 8

Parkurda aracın nasıl davranışlar sergileyeceğini gösteren karar verme algoritmaları yukardaki resimdeki gibidir. Yukarıdaki tabloda karar algoritması aracın iki şeridi ortalayarak hareketi , durması ,durak ve park yapmasını kapsamaktadır. Zed 2 kamera ve Lidar sensör verileri aracın karar verme algoritmalarını sağlayacaktır. Araç aldığı girdilere göre hız, fren ve direksiyonunun açısı değerini çıktı olarak göndermektedir.

7. Güvenlik Önlemleri

7.1. Dönüş Güvenlik Önlemleri

Araç Nesne Takibi ile ayarlanan trafik levhalarının verileriyle beraber dönüşler yapabilmektedir. Bu dönüşler sürecinde yaşanabilecek hızla ilgili problemleri çözmek adına aracın dönüşteki hızı, aracın düz gittiği durumdaki hızının yarısı olarak belirlenmiştir.

7.2.Şerit Tespiti Güvenlik Önlemi

Eğer araç şerit takibi yapamaz durumda olursa, yani şerit bulamaz ya da göremez olursa ani fren yapıp ve yeniden şerit bulana kadar ilerlemeyi durdurur.

7.3.Nesne Takibi Güvenlik Önlemleri

Araçtaki Zed 2 kamera derinlik algısı yapabilen bir kamera olduğundan, kamera ile nesneler arasındaki mesafe 1 metreden aşağı düşerse araç ani fren yapar ve tamamen durur.

7.4.Donanımsal Güvenlik Can Protokolü

Can Protokolünü diğer protokollerden ayıran en önemli farklar öncelikli ve 2 yönlü geçişlerdir. Bizde bu öncelikli ve 2 yönlü geçişleri daha öncelikli durumlar (yukarıda bahsedilen ani fren durumları vs.) olması halinde iletim için kullanmaktayız.

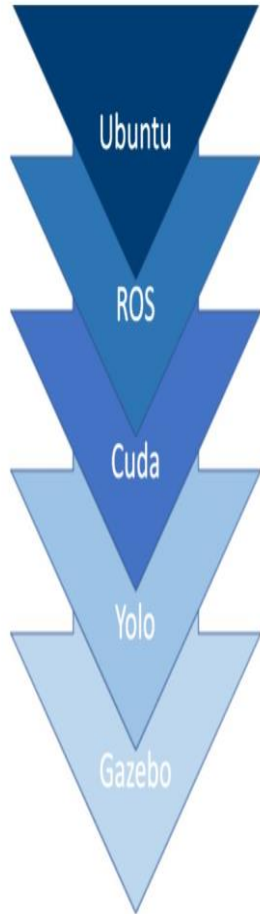
7.5.Araç üzerindeki güç kesiciler

Eğer araç beklenmedik bir faaliyette bulunursa aracın önünde ve arkasında yer alan güç kesme (acil durum) butonları sayesinde durdurulmaktadır. Aşağıdaki resimde acil durum butonları gösterilmiştir.



Resim 12-13

8. Simülasyon



- | |
|--|
| <ul style="list-style-type: none"> • Ubuntu 20.04 versiyonu kullanılmaktadır. • Performas iyileştirmeleri, güvenlik yamaları gibi güncellemelerle çalışma ortamının rahatlığı sebebiyle tercih edilmiştir. |
| <ul style="list-style-type: none"> • ROS noethic versiyonu kullanılmaktadır. • Açık kaynaklıdır • Python 3 desteği ve Gazeboya uyumu sebebiyle kullanılmıştır. |
| <ul style="list-style-type: none"> • CUDA kullanılmıştır. • Algoritmaların aynı anda birbirleriyle uyumlu işlem yapma özelliği kazandırmıştır. |
| <ul style="list-style-type: none"> • YOLO V5 versiyonu kullanılmaktadır. • Eğitim süresi karşılaştırması ve çıkarım zamanı diğer versiyon ve rakiplerine göre daha kullanışlıdır. |
| <ul style="list-style-type: none"> • Simülasyon ortamı olarak Gazebo kullanılmıştır. • Ros kütüphanesi uyumluluğu sebebiyle tercih edilmiştir. |

Tablo 9

8.1. Gazebo

Teknofest tarafından verilen modelin uyumluluğu, sensör ekleme kolaylığı ve gerçek dünyadaki gibi gürültülü veri üretme özelliklerinden dolayı Gazebo simülasyon ortamıyla çalışmalara devam edilmesine karar verilmiştir. Gazebo simülasyon ortamı ROS mimarisiyle uyumlu çalışan bir simülasyon ortamıdır. Çalışmalar Ubuntu 20.04 işletim sistemi üzerinden Linux altyapısını kullanarak gerçekleştirilmiştir. İşletim sistemi üzerinden araç ile haberleşmenin sağlanabilmesi ve algoritmaların geliştirilmesi için ROS mimarisi tercih edilerek kurulum yapılmıştır. Kurulumlar tamamlandıktan sonra ROS mimarisiyle birlikte Gazebo simülasyon ortamı da gelmektedir. Harita tasarımını Gazebo simülasyon ortamına uygun olan formatta (.dae) çıktı alınarak simülasyon ortamına aktarım işlemi gerçekleştirilmiştir

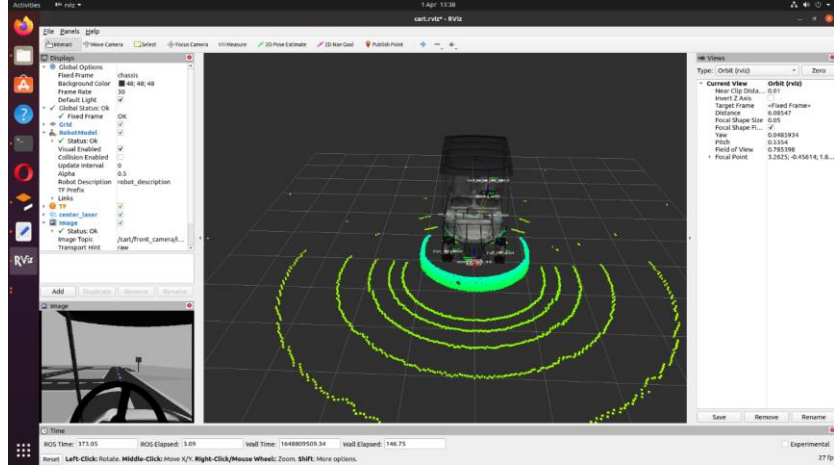
Gazebo, robot gelişimi için gerekli olan üç boyutlu simülasyon için robotlar, sensörler, çevre modelleri sağlayan ve fizik motoru ile gerçekçi simülasyonlar sunan bir simülatördür. Bu simülasyon ortamı hem kapalı mekanlar hem de açık mekanlar için kullanılan açık kaynak kodlu bir ortamdır. Gazebo'nun temeline bakıldığında istemci sunucu ilişkisiyle çalışmaktadır. Gzserver somut işlevleri yaparken, Gzclient kullanıcının isteklerinin yerine getirilmesi ve simülasyonun görsel aktarılmasını sağlar. ROS, bağımsız Gazebo ortamında bağıntılar sağlayan ve farklı ROS bileşenleriyle entegrasyon sağlayabilen gazebo_ros_pkgs adlı bir paket kullanır. Bu paketler, ROS mesajları, konuları ve hizmetleri kullanarak Gazebo da bir robotu simüle etmek için gerekli arayüzleri sağlar.

8.2. ROS

ROS, bilgisayar üzerinden robot bileşenlerini kontrol eden BSD lisanslı açık kaynak kodlu kavramsal bir işletim sistemidir. Dolayısıyla gerçek bir işletim sisteminde çalışmalıdır. Şu an için yalnızca LINUX tabanlı sistemler üzerinde çalışılmaktadır. ROS, donanım soyutlaması, aygıt sürücüler, birden çok makine üzerinden işlemler arasındaki iletişim, test etme ve görselleştirme araçları ve çok daha fazlası için işlevsellik sağlar. ROS mimarisi üç bölüme veya kavram seviyesine ayrılmıştır. Bunlar; dosya sistemi seviyesi, hesaplama grafiği seviyesi ve topluluk düzeyi olarak sınıflandırılabilir. ROS da bulunan paketler , gerçek donanımın kesin özelliklerine sahip üç boyutlu bir robot modeli tanımlamasını oluşturmamıza yardımcı olur.

8.3. Simülasyon Ortamı

Blender programı üzerinden yollar tasarlandı ve haritada kullanılmak üzere uygun hale getirildi. Ayrıca arabanın ilerleyeceği ortamın gerçek dünyayla birebir aynı olması için hava şartları, atmosfer, levha ve araba boyutu tamamıyla gerçeğe uygun olarak tasarlanmış olup ortama ona göre entegre edilmiştir. Blender Modelleme Programını kullanarak trafik levhalarını gerekli renk, boyut ve şekillerde oluşturup '.dae' formatında simülasyon ortamına eklenmiştir. Teknofest tarafından gönderilen araçta simülasyona entegre edilerek simülasyon çalıştırılabilir hale getirilmiştir. ROS kullanılarak araç üzerindeki sensörlerin verileri ilgili topiclerden yayınlanmıştır. Algoritmaların oluşturulması esnasında bu topiclere abone olunarak sensör verilerinin elde edilmesi ve işlenmesi sağlanmıştır.



Resim 14

9. Sistem Entegrasyonu

Simülasyon çalışmalarımızda Ubuntu 20.04 ile ROS Noetic altyapısı kullanılmıştır. Araç üzerindeki sensör ve kameradan gelen bilgileri kullanarak algoritmamızı yazdık. Araç hareketi CAN BUS'a komut göndererek sağlanmaktadır. Nesne tespit yöntemi olarak YOLOv5 modelinin “Trafik İşareti Tespit ve Tanıma Algoritması” olarak kullanılmaktadır. Takımımız tarafından oluşturulmuş ve etiketlenmiş olan, trafik işaretleri ve ışıkları bulunduran veri seti ile eğitildi. Araç kamerası Zed2’den görüntülenen levhaların bilgileri toplanarak kullandığımız algoritma ile araç güzergahı belirlendi. Aracın ön tarafında bulunan ve aracın etrafındaki engelleri algılamasını ve uzaklık mesafesini ölçmesini sağlayan Lidar, gelen verilerle aracın engellerden kaçmasına olanak sağladı.

10. Test ve Doğrulama

Teknofest tarafından bize sunulan test imkanlarıyla hazır araç üzerinde testler yapılmıştır. Aracımızın CAN BUS protokolüyle tekerleklerine iletişim sağlayıp sağlanmadığı test ederek hız, fren, tekerlek açısı gibi aracın hareketine etki eden faktörler bu protokol üzerinden test edilmiştir. Test için bir takım teknik problemler de yaşanmıştır. Geliştirdiğimiz algoritmaları araç üzerinde tam anlamıyla test etme imkanı bulamadık. Test ve yazılım kısmında iyileştirmelerimiz devam etmektedir.

Araç üzerinde sırasıyla bu testlerin yapılması ön görülmektedir.

- Aracın yol şeritlerini algılayıp onlara göre yolu ortalayabilmesi
- Fren ve Ani Fren gerçekleşmesi
- Veri setinin oluşturulması ve nesne tanımlama algoritmasının kontrolü
- Uzaktan aracı acil ve normal durdurma müdahale edilmesi
- Aracın duraktan yolcu alıp indirmesi
- Park kısmında aracın nesne tanımlama ile uygun alana park yapması

Test için halen zamanımız olup bu konuda çalışmalarımız devam etmektedir.

11. Referanslar

- [1] https://www.sas.com/tr_tr/insights/analytics/yapay-zeka-nedir.html#world
- [2] https://www.sas.com/tr_tr/insights/analytics/neural-networks.html
- [3] [https://www.yapayzekatr.com/2019/12/16/derin-ogrenmenedir/#:~:text=Yapay%20zeka%20alan%C4%B1%2C%20makinelerin%20deneyiml e,bir%20 makine%20%C3%B6%C4%9Freniminin%20alt%20k%C3%BCmesidir. \(3\)](https://www.yapayzekatr.com/2019/12/16/derin-ogrenmenedir/#:~:text=Yapay%20zeka%20alan%C4%B1%2C%20makinelerin%20deneyiml e,bir%20 makine%20%C3%B6%C4%9Freniminin%20alt%20k%C3%BCmesidir. (3))
- [4] <https://medium.com/@tuncerergin/convolutional-neural-network-convnet-yada-cnn-nedir-nasil-calisir-97a0f5d34cad>
- [5] <https://www.smartera.com.tr/gercek-zamanli-nesne-takibi-real-time-object-detection-wyolo-python/>
- [6] <https://cdn.t3kys.com/media/upload/userFormUpload/G2bMObqJpDyk5GjnzC5p9cABAlciic8.pdf>
- [7] <https://cdn.t3kys.com/media/upload/userFormUpload/DK0BK1tlcp2ktP1J2htmAxFB8qV177Sn.pdf>
- [8] <https://www.hurriyet.com.tr/egitim/sensor-nedir-nasil-calisir-ve-nerelerde-kullanilir-sensor-cesitleri-hakkinda-bilgi-41720326>
- [9] <https://tr.wikipedia.org/wiki/LIDAR>
- [10] <https://ioturkiye.com/2021/06/otonom-araclarda-perception/>
- [11] <https://openzeka.com/urun/zed-stereo-camera/>
- [12] <https://cbsakademi.ibb.istanbul/imu-nedir/>
- [13] <https://paperswithcode.com/task/object-detection>
- [14] https://en.wikipedia.org/wiki/Object_detection
- [15] https://en.wikipedia.org/wiki/Line_detection
- [16] <https://towardsdatascience.com/lines-detection-with-hough-transform-84020b3b1549>
- [17] <https://roboflow.com/>
- [18] https://en.wikipedia.org/wiki/PID_controller